UNIVERSIDAD POLITÉCNICA ESTATAL DEL CARCHI

POSGRADO

MAESTRÍA EN INGENIERÍA DE SOFTWARE

INFORME · ACTIVIDAD AUTÓNOMA · SEMANA 2 · MÓDULO 8

AUTOMATIZACIÓN DE PROCESO DE SOFTWARE

Informe y artefactos de la cadena CASE

El caso de FacturaEC, sistema web de facturación electrónica

Integrantes:

Byron Bravo

Jairo Jácome

Jordan Montenegro

Luis Cisneros

Docente:

MsC. José Luis Rodríguez

Tulcán, Ecuador

Octubre de 2026

Informe y artefactos de la cadena CASE de FacturaEC

Introducción

Este informe documenta la cadena de herramientas CASE (ingeniería de software asistida por computador) que el equipo construyó sobre FacturaEC, el sistema web de facturación electrónica para pequeñas empresas de Ecuador cuyo dossier de arquitectura se entregó en la versión v0.1.0. El objetivo de la actividad es organizar tareas de alto volumen mediante herramientas CASE y aplicar buenas prácticas de automatización. Para ello se elaboró el mapa de toolchain del proceso, se derivaron de los escenarios del dossier historias con criterios en Gherkin, se escribió un contrato OpenAPI del que se generaron la documentación, el mock y los stubs, y se construyó un esqueleto ejecutable con pruebas que nacen de esos criterios. Por último, se evaluaron y seleccionaron las herramientas con criterios explícitos, siguiendo ISO/IEC 14102 (ISO/IEC, 2008), y se verificó de forma crítica todo lo que produjo la inteligencia artificial.

Todos los artefactos están en el repositorio del proyecto (https://github.com/byronbravo1080/ApiFacturacion), en la versión etiquetada v0.2.0, con commits convencionales (Chacon & Straub, 2014). El borrador inicial del contrato se conserva en docs/evidencias/openapi-v1-borrador.yaml como referencia histórica de los defectos detectados y corregidos durante la validación, y las evidencias de ejecución de cada herramienta se presentan en las secciones correspondientes de este informe.

Mapa de toolchain y priorización de tareas de alto volumen

Proceso del proyecto por fase

El análisis cambia de objeto respecto del dossier: ya no se estudia el producto, sino el proceso para construirlo y verificarlo. Se usan las seis etapas de la práctica del módulo, alineadas con los procesos técnicos de ISO/IEC/IEEE 12207 (ISO/IEC/IEEE, 2017). Con

cuatro integrantes, los roles rotan entre hitos. La Tabla 1 resume las etapas, y el detalle con su diagrama está en `docs/06-mapa-toolchain.md`.

Tabla 1

Etapas del proceso de FacturaEC, artefactos y responsables

Etapa	Artefactos	Rol responsable
Analizar	Historias HU con criterios CA en Gherkin; matriz de trazabilidad	Arquitecto
Diseñar	Vistas C4, ADR y contrato OpenAPI (`api/openapi.yaml`)	Arquitecto y responsable del contrato
Construir	Stubs generados y esqueleto del monolito modular (`src/`)	Equipo de desarrollo
Probar	Pruebas BDD de E1, E2 y E6, conformidad con el contrato y análisis estático	Responsable de calidad
Integrar y revisar	Pull requests con revisión cruzada y rúbrica	Responsable de repositorio
Entregar	Versión etiquetada (SemVer), `CHANGELOG.md` y documentación	Responsable de repositorio

Nota. Los roles son ilustrativos; una persona puede asumir más de uno.

Diez tareas puntuadas por frecuencia, esfuerzo y riesgo

Se listaron diez tareas concretas del proceso y se puntuaron de 1 a 5 en frecuencia (F), esfuerzo hecho a mano (E) y riesgo si fallan (R). En la escala de riesgo, 5 significa que el fallo rompe directamente E1, E2 o E6 (las prioridades del dossier que entran en esta versión) sin que otra barrera lo detecte, y 3 que causa retrabajo o un defecto que otra barrera detecta. El puntaje es el producto de los tres valores; Alta va desde 60, Media de 20 a 59 y Baja hasta 19. Cada valor se justificó con un hecho observable en `docs/06-mapa-toolchain.md`. Por ejemplo, T7 tiene esfuerzo 5 porque E6 exige 1000 emisiones concurrentes contra PostgreSQL, imposibles de reproducir a mano, y riesgo 5 porque un secuencial duplicado es un comprobante que el SRI rechaza.

Tabla 2

Priorización de las diez tareas (frecuencia × esfuerzo × riesgo)

ID	Tarea	F	E	R	Puntaje	Prioridad
T7	**Pruebas de aceptación de E6, E1 y E2**	4	5	5	**100**	**Alta**
T4	**Validar el contrato OpenAPI con un linter**	4	4	4	**64**	**Alta**
T6	**Generar stubs desde el contrato**	4	4	3	**48**	Media
T5	Documentación y mock desde el contrato	4	3	3	36	Media
T9	Revisar pull requests con rúbrica	4	3	3	36	Media

ID	Tarea	F	E	R	Puntaje	Prioridad
T8	Linter y análisis estático del código	5	2	3	30	Media
T2	Trazabilidad historia → escenario → criterio → endpoint → ADR → prueba	3	3	3	27	Media
T1	Criterios de aceptación en Gherkin	3	3	2	18	Baja
T3	Actualizar las vistas C4	2	3	2	12	Baja
T10	Changelog y número de versión	2	2	2	8	Baja

Nota. Las tres tareas priorizadas (en negrita) son las de mayor puntaje. T1, T2, T5, T8 y T9 se ejecutaron como tareas habilitantes porque la cadena las necesita; T3 no hizo falta y T10 se difiere a la sesión 10.

Herramienta y categoría por fase

A cada fase se le asignó una herramienta para su tarea de mayor puntaje. La categoría depende de la función que la herramienta cumple en esa fase: Upper CASE en análisis y diseño, Lower CASE en construcción y pruebas, e I-CASE cuando ambas se integran sobre un repositorio común. Las herramientas obligatorias son gratuitas y de código abierto; la única de pago, la IA, es un complemento opcional. La Figura 1 muestra cómo se encadenan.

Tabla 3

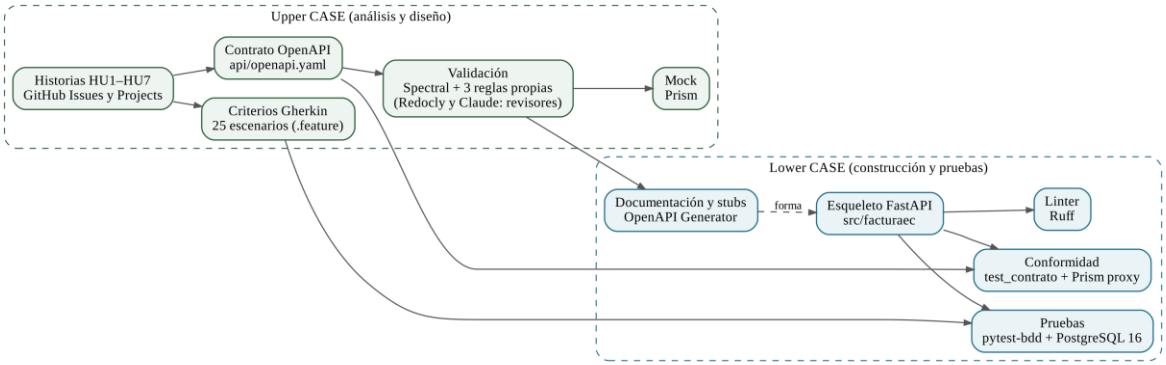
Herramienta candidata por fase

Fase	Tarea (puntaje)	Herramienta	Categoría
Analizar	T2 (27)	GitHub Issues y Projects; Gherkin en archivos `.feature`	Upper CASE
Diseñar	T4 (64)	Swagger Editor, Spectral (linter) y Prism (mock)	Upper CASE
Construir	T6 (48)	OpenAPI Generator 7.26.0 (`python-fastapi` y `html2`)	Lower CASE
Probar	T7 (100)	pytest 9.1 + pytest-bdd 9.0 sobre PostgreSQL 16, con Claude Opus 5.5 como asistente	Lower CASE
Integrar y revisar	T9 (36)	GitHub pull requests con plantilla de rúbrica	Lower CASE
Entregar	T10 (8)	Git (etiqueta SemVer) y `CHANGELOG.md`; automatización en la sesión 10	Lower CASE

Nota. La herramienta de IA cambia de categoría según su función: es Upper CASE cuando revisa el contrato o redacta criterios, y Lower CASE cuando escribe pruebas. GitHub, visto como un todo, funciona como I-CASE.

Figura 1

Cadena CASE de FacturaEC: de las historias a las pruebas



Nota. Elaboración propia con Graphviz. La línea discontinua indica que de los stubs se toma la forma (rutas y códigos), no la implementación.

Prueba de las herramientas de las tres tareas priorizadas

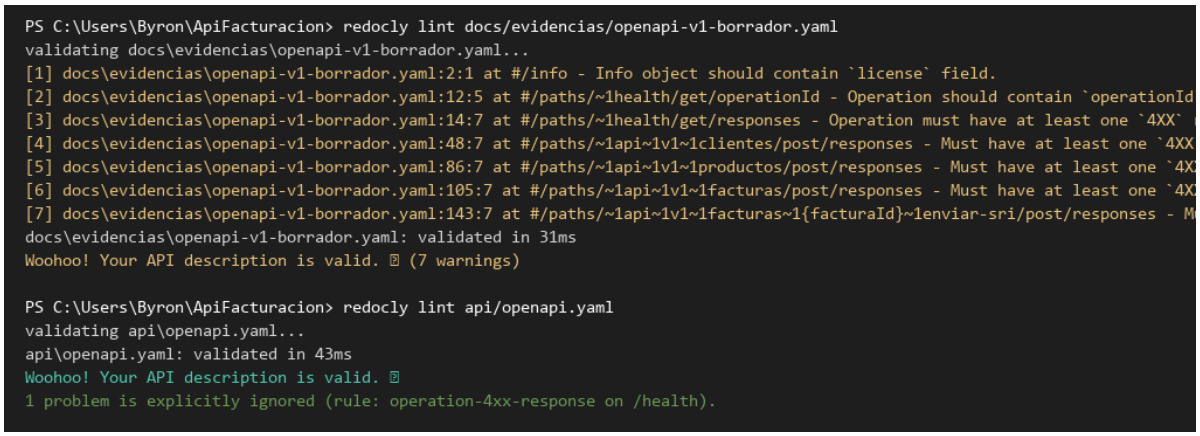
Las herramientas de T7, T4 y T6 se instalaron y probaron sobre los artefactos reales del proyecto, no sobre ejemplos genéricos. Para T4 se probaron tres alternativas sobre el mismo borrador del contrato, entre ellas una de IA. La Tabla 4 resume lo que funcionó y lo que falló en cada caso; los registros completos están en la sección 6 de `docs/06-mapa-toolchain.md`.

Tabla 4

Resultado de la prueba de cada herramienta

Tarea y herramienta	Qué funcionó	Qué falló o se detectó
T4 · Spectral 6.17.0	17 advertencias en el borrador v1; 0 hallazgos de cualquier severidad en la versión final	No vio que el registro y `/health` heredaban el token obligatorio, error que impedía registrarse (CA1.1)
T4 · Redocly CLI 2.71.0	8 advertencias, entre ellas 5 operaciones sin respuestas 4xx; versión final válida	Tampoco vio el token heredado; envía telemetría por defecto (se desactivó)
T4 · Claude Opus 5.5 como revisor (IA)	18 hallazgos (17 reales y 1 parcialmente correcto), incluidos los dos errores más graves; la repetición coincidió en los 6 errores	1 hallazgo parcialmente incorrecto, repetido en las dos ejecuciones; no garantiza la sintaxis
T6 · OpenAPI Generator 7.26.0	50 archivos generados; el stub arranca y expone las 8 rutas del contrato	Responde 500 «Not implemented», no entrega el token a la implementación y aceptó un token falso
T7 · pytest-bdd + PostgreSQL 16	Ejecuta el Gherkin en español; 31 pruebas aprobadas, con 1000 emisiones enviadas desde 50 hilos concurrentes, sin duplicados ni saltos	Su primera ejecución detectó un secuencial duplicado bajo concurrencia, que con SQLite habría pasado
T7 · Claude Opus 5.5 como asistente (IA)	Escribió los pasos de las pruebas y el esqueleto	El defecto anterior estaba en su propio código; una prueba de mutación confirmó que la prueba sí detecta la falta del bloqueo

Validación del contrato con Redocly CLI (Tarea T4)



Nota. Salida de Redocly CLI mostrando 7 advertencias en el borrador v1 y validación exitosa (0 errores) sobre el contrato final api/openapi.yaml.

Cadena funcionando: historias, contrato, documentación, stubs y pruebas

Historias y criterios de aceptación en Gherkin

Se redactaron siete historias de usuario con el formato «Como / quiero / para» (Cohn, 2004), cada una ligada a un stakeholder, un driver y un escenario del dossier. Sus criterios se escribieron en Gherkin en español, con la medida numérica del escenario de origen y un solo comportamiento por escenario (Cucumber, s. f.; North, 2006). El resultado son 25 escenarios ejecutables. Las historias pasaron la revisión INVEST y la lista de atributos de ISO/IEC/IEEE 29148 (ISO/IEC/IEEE, 2018); al aplicar esa lista se precisaron tres términos ambiguos: factura conservada, recurso ajeno y salto. La Tabla 5 muestra la trazabilidad, y el detalle está en `docs/07-historias-y-criterios.md`.

Tabla 5

Trazabilidad de las historias hasta el endpoint, el ADR y la prueba

Historia	Escenario	Criterios	Operación del contrato	ADR
HU1 Registrar el negocio	E1	CA1.1, CA1.2	`registrarEmisor`	ADR-001, ADR-005
HU2 Clientes visibles solo para el emisor	E1	CA2.1 a CA2.3	`registrarCliente`, `obtenerCliente`	ADR-001, ADR-002
HU3 Productos con tarifa de IVA	E7 (indirecto)	CA3.1, CA3.2	`registrarProducto`	ADR-001

Historia	Escenario	Criterios	Operación del contrato	ADR
HU4 Totales con IVA	E3 (indirecto)	CA4.1	`crearFactura`	ADR-001, ADR-003
HU5 Secuencial único y clave de acceso	E6	CA5.1 a CA5.3	`crearFactura`	ADR-002, ADR-003
HU6 Envío al SRI sin perder facturas	E2	CA6.1 a CA6.4	`enviarFacturaSri`	ADR-003
HU7 Consultar el estado (contador)	E2	CA7.1	`obtenerFactura`	ADR-003

Nota. E8 (lentitud del SRI), la descarga del XML y del RIDE, el inicio de sesión, la nota de crédito y la primera factura desde el frontend quedan como historias HU8 a HU12 del backlog para los hitos posteriores.

El criterio más exigente es CA5.3, que traduce literalmente la medida de E6:

```
@HU5 @E6 @CA5.3 @postgresql
Escenario: Solicitudes concurrentes de emisión del mismo emisor
  Cuando se envían 1000 solicitudes concurrentes de creación de factura del emisor
"A"
  Entonces se crean 1000 facturas
  Y hay 0 secuenciales duplicados
  Y hay 0 saltos en la numeración
```

La prueba envía las 1000 solicitudes desde 50 hilos simultáneos y la API las atiende con hasta 40 hilos: es concurrencia real, aunque acotada. Basta para que, sin el bloqueo de fila, aparezcan secuenciales duplicados, como demuestra la prueba de mutación.

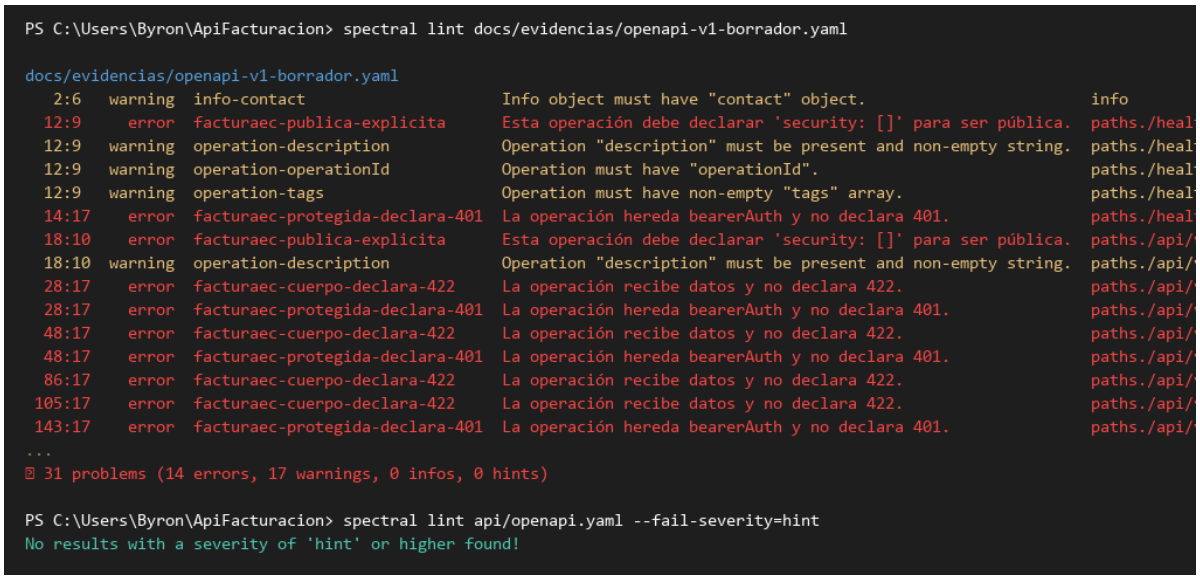
Contrato OpenAPI y su validación

El contrato `api/openapi.yaml` se escribió antes que el código (API-first) y es la fuente de verdad de la cadena (OpenAPI Initiative, s. f.). Esa decisión quedó registrada en el ADR-005, que modifica al ADR-004: antes el contrato se iba a generar desde el código. Define siete operaciones de negocio más `/health`, con respuestas de error comunes (401, 404, 409 y 422), un esquema de error, montos con dos decimales y la clave de acceso de 49 dígitos. No contiene datos de clientes ni de contribuyentes reales; solo los nombres de los autores en el contacto.

El borrador v1 se revisó con las tres alternativas de T4. Spectral solo encontró advertencias de estilo y Redocly agregó las operaciones sin respuestas 4xx. Solo la revisión con IA detectó que el `security` global se heredaba en el registro y en `/health`, lo que impedía registrarse. Tras aplicar las correcciones, Spectral no reporta hallazgos de ninguna

severidad. Para que ese control no dependa de repetir la revisión con IA, los hallazgos recurrentes se convirtieron en cuatro reglas propias de Spectral (`.spectral.yaml`; Stoplight, s. f.): toda operación protegida declara 401, el registro y `/health` se declaran públicos de forma explícita, ninguna otra operación puede redefinir la seguridad, y toda operación con cuerpo declara 422. Aplicadas al borrador v1, esas reglas dan 14 errores; sobre el contrato final, ninguno. Una prueba negativa, que vuelve pública una operación protegida, hace fallar la regla correspondiente.

Validación con Spectral: Borrador v1 frente a Contrato Final v2 (Tarea T4)



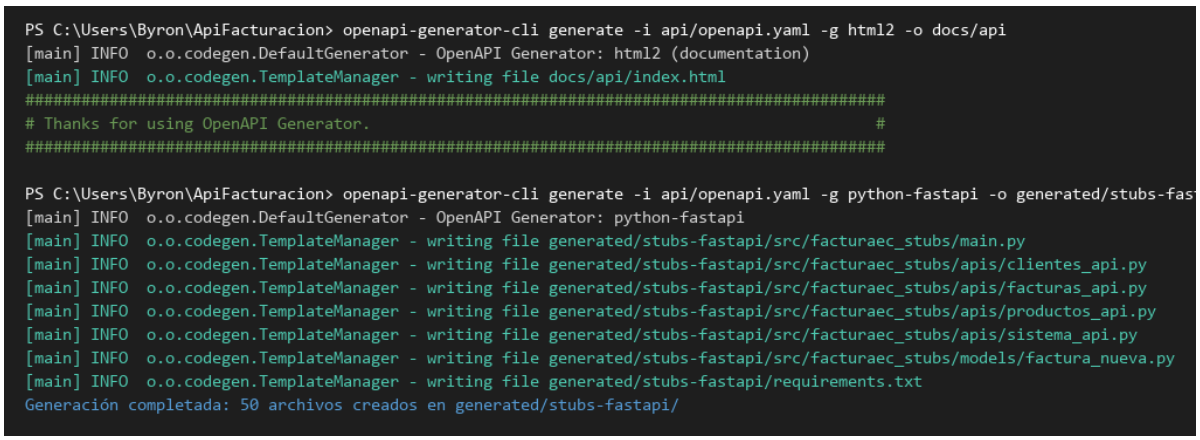
Nota. Reglas propias de .spectral.yaml aplicadas al borrador v1 (31 problemas: 14 errores y 17 warnings) y verificación sin errores en api/openapi.yaml.

Documentación, mock y stubs generados

Desde el contrato validado se generaron tres derivados. La **documentación** HTML se generó con OpenAPI Generator (OpenAPI Generator, s. f.) en `docs/api/index.html`, y la API publica el mismo contrato en `/docs`. El **mock** de Prism responde 401 sin token, 201 con datos válidos y 422 ante una tarifa de IVA no permitida. En su primera versión todos los errores salían con el mismo ejemplo `NO_ENCONTRADO`. Eso llevó a agregar ejemplos por tipo

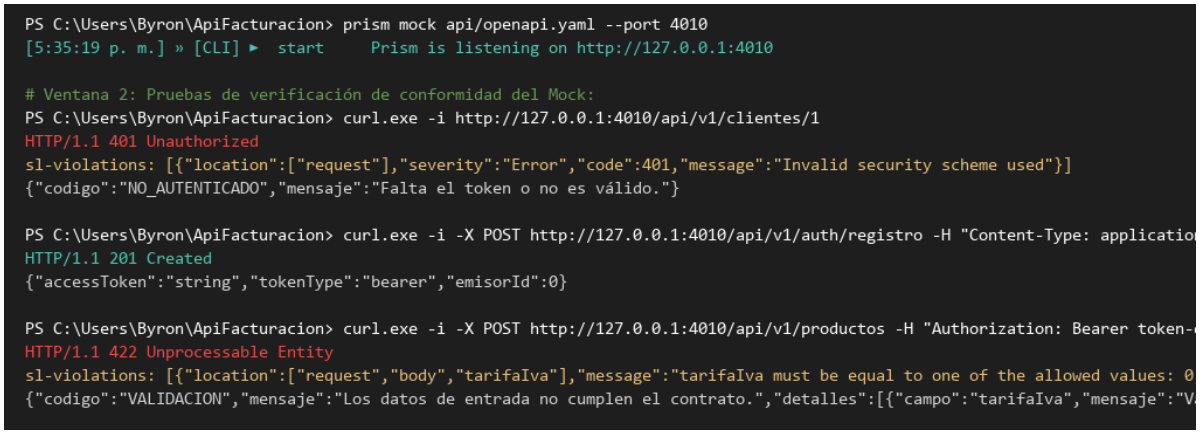
de error, a separar el 409 del RUC duplicado del 409 de la factura ya autorizada y a dar a la factura un ejemplo con totales coherentes. Su límite es que devuelve ejemplos estáticos, no el detalle real de cada validación. Los **stubs** en Python y FastAPI se generaron en `generated/stubs-fastapi/`. El stub arranca y expone las ocho rutas, pero no entrega el token a la implementación. Por eso se decidió no editarlo: se toma de él la forma (rutas, códigos y nombres de operación), y la implementación vive en `src/`.

Generación de documentación HTML y stubs FastAPI con OpenAPI Generator (Tareas T5 y T6)



Nota. Ejecución de openapi-generator-cli generando docs/api/index.html y los 50 archivos de stubs estructurados en generated/stubs-fastapi/.

Servidor Mock en vivo con Stoplight Prism (Tarea T5)



Nota. Prism respondiendo HTTP 401 ante petición sin token, HTTP 201 en registro exitoso y HTTP 422 ante tarifa de IVA inválida (12).

Esqueleto ejecutable y pruebas derivadas de los criterios

El esqueleto en `src/facturaec/` implementa las ocho operaciones respetando el monolito modular del ADR-003:

- **E1:** filtro por emisor en cada consulta, con el mismo 404 para lo inexistente y lo ajeno.
- **E6:** asignación del secuencial bajo bloqueo de fila y restricción única (ADR-002), y clave de acceso con módulo 11.
- **E2:** adaptador `SriGateway` simulado, con tres intentos y esperas de 1 s y 2 s.

Las pruebas usan pytest-bdd, que ejecuta los mismos archivos `.feature` sin traducirlos. Tres pruebas adicionales verifican el contrato frente al código (las mismas operaciones, al menos cuatro endpoints de negocio y el contrato publicado en `/docs`), y otras tres comprueban que `docs/07` muestre el mismo Gherkin que se ejecuta. La Tabla 6 resume los resultados.

Tabla 6

Resultados de la ejecución de la cadena

Paso	Herramienta	Resultado
Contrato	Spectral (`spectral:oas` + 4 reglas propias)	0 hallazgos de cualquier severidad; la prueba negativa de las reglas falla, como se espera
Contrato (alternativa)	Redocly CLI	Válido; 1 excepción justificada (`/health` sin 4xx) y 1 regla desactivada (servidor `localhost`)
Pruebas con SQLite	pytest + pytest-bdd	30 aprobadas, 1 omitida (CA5.3 requiere PostgreSQL)
Pruebas con PostgreSQL 16	pytest + pytest-bdd	31 aprobadas en unos 13 s; CA5.3: 1000 solicitudes desde 50 hilos, 0 duplicados, 0 saltos
Prueba de mutación	pytest (CA5.3 sin `with_for_update()`)	Falla, como se esperaba: la prueba detecta la falta del bloqueo
Conformidad con el contrato	Prism en modo proxy (`--errors`)	1 violación en la primera ejecución (`direccion: null`, en 2 llamadas); 0 en 10 llamadas tras corregirla
Análisis estático	Ruff 0.17.2	38 hallazgos en la primera ejecución; 0 al final

Nota. Con SQLite la prueba de E6 se omite a propósito: el ADR-002 advierte que SQLite ignora el bloqueo de fila.

Ejecución del suite de pruebas BDD con pytest (Tarea T7)

```
PS C:\Users\Byron\ApiFacturacion> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.4, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\Byron\Desktop\MAESTRIA UPEC\...\ApiFacturacion, configfile: pyproject.toml
plugins: anyio-4.15.1, bdd-9.0.0
collected 31 items

tests/step_defs/test_envio_sri.py::test_envío_con_el_sri_disponible PASSED           [  3%]
tests/step_defs/test_envio_sri.py::test_envío_con_el_sri_no_disponible PASSED        [  6%]
tests/step_defs/test_envio_sri.py::test_reenvío_cuando_el_sri_se_restablece PASSED   [  9%]
tests/step_defs/test_envio_sri.py::test_rechazo_del_reenvío_de_factura_autorizada PASSED [ 12%]
tests/step_defs/test_facturacion.py::test_registro_producto_tarifa_iva_válida PASSED [ 19%]
tests/step_defs/test_facturacion.py::test_rechazo_tarifa_iva_no_permitida PASSED     [ 22%]
tests/step_defs/test_facturacion.py::test_cálculo_linea[10.00-15-1-10.00-1.50-11.50] PASSED [ 25%]
tests/step_defs/test_facturacion.py::test_secuenciales_consecutivos PASSED          [ 38%]
tests/step_defs/test_facturacion.py::test_clave_de_acceso_válida PASSED             [ 41%]
tests/step_defs/test_facturacion.py::test_solicitudes_concurrentes_mismo_emisor SKIPPED [ 45%]
tests/step_defs/test_registro_y_aislamiento.py::test_registro_emisor_nuevo PASSED   [ 48%]
tests/step_defs/test_registro_y_aislamiento.py::test_otro_emisor_no_puede_ver_recurso_ajeno PASSED [ 58%]
tests/step_defs/test_registro_y_aislamiento.py::test_solicitud_sin_token[POST /facturas] PASSED [ 74%]
tests/test_contrato.py::test_el_codigo_implementa_todas_y_solo_las_operaciones PASSED [ 83%]
tests/test_contrato.py::test_el_contrato_tiene_al_menos_cuatro_endpoints_de_negocio PASSED [ 87%]
tests/test_contrato.py::test_docs_publica_el_contrato_y_no_el_esquema_inferido PASSED [ 90%]
tests/test_documentacion.py::test_docs07_contiene_el_gherkin_ejecutable PASSED      [100%]
========================= short test summary info =========================
SKIPPED [1] scenario.py:305: CA5.3 requiere PostgreSQL: defina TEST_DATABASE_URL (ADR-002)
================= 30 passed, 1 skipped, 1 warning in 4.08s ==================
```

Nota. Resultado de pytest -v ejecutando los 25 escenarios Gherkin y 6 pruebas de contrato y sincronía documental (30 aprobadas, 1 omitida por SQLite).

Análisis estático y formato de código con Ruff (Tarea T8)

```
PS C:\Users\Byron\ApiFacturacion> ruff check src tests
All checks passed!

PS C:\Users\Byron\ApiFacturacion> ruff format --check src tests
20 files already formatted
```

Nota. Verificación de análisis estático (ruff check) y estilo (ruff format) con 0 errores sobre el código de src/ y tests/.

Matriz de evaluación y selección de herramientas

Se aplicaron los cuatro pasos de ISO/IEC 14102: definir requisitos, evaluar capacidades, comparar opciones y decidir (ISO/IEC, 2008). La función evaluada es revisar y validar el contrato (T4), porque es la puerta de la cadena y porque admite alternativas que reciben la misma entrada y producen hallazgos que se pueden contar. Los criterios y sus pesos están en la Tabla 7. Cada puntaje se tomó de la prueba realizada; el detalle está en `docs/08-matriz-seleccion.md`.

Tabla 7

Matriz ponderada de selección (puntajes de 1 a 5)

Criterio	Peso	Spectral	Redocly CLI	Claude Opus 5.5 (IA)
C1 Capacidad de detección	30	1	3	5
C2 Fiabilidad y reproducibilidad	20	5	5	3
C3 Integración en la cadena	20	5	3	1
C4 Facilidad de adopción	10	4	4	3
C5 Costo y licencia	10	5	5	1
C6 Riesgo de uso y privacidad	10	5	3	1
Total ponderado (sobre 5)	**100**	**3,70**	**3,70**	**2,80**

Nota. Total = Σ(peso × puntaje) / 100. Cada puntaje corresponde a un nivel de la rúbrica de 1 a 5 de `docs/08`. Por ejemplo, Spectral obtiene 1 en C1 porque con su conjunto estándar solo detectó estilo, y Redocly 3 en C6 porque envía telemetría que se puede desactivar. Para C2 de la IA se repitió la revisión: coincidieron los 6 errores, varió el número de hallazgos (18 y 16) y se repitió el mismo hallazgo parcialmente incorrecto.

Con los pesos base, Spectral y Redocly empatan en 3,70 y la IA queda en 2,80. Con pesos iguales, o si pesan más la fiabilidad y la integración, gana Spectral. Si la detección pesa la mitad, Spectral queda último (2,90), y Redocly y la IA empatan (3,50). Ese análisis de sensibilidad muestra que ninguna herramienta sola cubre bien la detección: los linters son fiables pero no ven el significado, y la IA ve el significado pero no es reproducible ni bloquea un pipeline. Antes de decidir se declaró una regla de desempate: primero decide un requisito obligatorio y, si el empate persiste, el criterio de mayor peso. La actividad exige validar con Spectral, así que el empate se resuelve a su favor. La decisión queda así:

- **Spectral se selecciona** como validación obligatoria, con las cuatro reglas propias, que mitigan su baja detección. No se genera nada si reporta errores.
- **Claude queda fuera de la cadena obligatoria**, porque es de pago y tiene el total más bajo. Se usa como complemento opcional para la revisión semántica en los cambios mayores del contrato, siempre verificada y sin datos sensibles. La revisión cruzada humana la sustituye si no hay acceso a la IA.
- **Redocly no se adopta**, porque empata con Spectral, su aporte diferencial (el aviso genérico de 4xx) queda cubierto con más precisión por las reglas propias, y envía telemetría por defecto.

Los riesgos de esta selección y su mitigación se registraron en la sección 4.1 del documento de la matriz. Los principales son aceptar un hallazgo plausible pero incorrecto, exponer información al servicio de IA y que las reglas propias queden desactualizadas.

Verificación de los artefactos generados y registro del uso de IA

La regla aplicada fue que ninguna salida de la IA se acepta sin una verificación por ejecución real o sin contrastarla con los criterios del dossier. Cada uso se registró en `docs/04-registro-ia.md` con la herramienta y su versión, el prompt resumido, la salida, los defectos, la decisión y la verificación. La Tabla 8 los resume.

Tabla 8

Defectos encontrados en lo generado con IA y cómo se verificaron

Artefacto generado	Defecto encontrado	Cómo se detectó	Decisión
Contrato v1	Registro y `/health` exigían token; faltaban 401, 404, 409 y 422	Revisión con IA independiente, contrastada con los linters	Modificar
Ejemplo de clave de acceso	Dígito verificador incorrecto	Cálculo del módulo 11 con un script	Modificar
Código del secuencial	Duplicados bajo concurrencia (lectura sin refrescar la fila bloqueada)	CA5.3 contra PostgreSQL; prueba de mutación	Modificar
Respuesta de clientes	`null` donde el contrato declara un texto	Prism en modo proxy	Modificar
Prueba de conformidad	Comparaba contra un conjunto vacío	La propia prueba falló	Modificar
Script de verificación	Ejecutaba PostgreSQL en el paso de SQLite	Conteo de pruebas aprobadas (31 en lugar de 30)	Modificar
Mapa y matriz	Dos cifras mal contadas; reproducibilidad de la IA puntuada sin medir	Conteo con un script; repetición de la revisión	Modificar
Historias y criterios	Ninguno al ejecutar; límite de 60 s medido con reloj simulado	Ejecución con pytest-bdd y lista de ISO/IEC/IEEE 29148	Aceptar con limitación declarada
Matriz, informe y documentos	Seis puntajes fuera del nivel de su rúbrica; IA de pago seleccionada pese al costo cero; cifras más fuertes que la evidencia	Revisión cruzada de consistencia entre la matriz, el mapa y la rúbrica (20 observaciones)	Modificar

La IA aportó más en la revisión semántica del contrato y falló donde más importaba: el código que escribió tenía un defecto de concurrencia que no se ve leyendo el código ni probando con SQLite, y que solo apareció con una prueba derivada del criterio de E6 y ejecutada contra la base real. Esto confirma la regla práctica del módulo: a mayor riesgo de la tarea, mayor supervisión humana y más ejecución real. No se ingresaron credenciales, URI de bases de datos, RUC reales ni datos personales en ninguna herramienta de IA, y los RUC de las pruebas son ficticios. La revisión del código generado frente a OWASP Top 10 (OWASP Foundation, s. f.) queda como tarea del hito siguiente.

Conclusiones

La cadena funciona de punta a punta de forma reproducible: historias, criterios, contrato validado, documentación, mock, stubs, esqueleto y 31 pruebas que pasan contra PostgreSQL. Priorizar con frecuencia × esfuerzo × riesgo concentró el trabajo donde rendía. La tarea de mayor puntaje, las pruebas de E6, E1 y E2, fue la que encontró el defecto más grave, y la segunda, la validación del contrato, evitó dos errores antes de escribir código.

La evaluación con ISO/IEC 14102 mostró que la selección no siempre es una sola herramienta. Los linters y la IA se complementan, y el mejor uso de la IA fue convertir sus hallazgos en reglas deterministas que corren sin ella. Quedan para los hitos siguientes la lentitud del SRI (E8), el XML firmado y el RIDE, el frontend y el pipeline de CI/CD, en el que Spectral, las pruebas con PostgreSQL y Ruff pasarán a ejecutarse en cada cambio.

Declaración de uso de IA

Herramienta: Claude Opus 5.5 (Anthropic), mediante claude.ai.

Propósito: asistencia en borradores del mapa de toolchain, las historias, el contrato y las pruebas; evaluación como revisor semántico del contrato en la matriz de selección; y revisión cruzada de consistencia documental.

Verificación: cada artefacto se verificó con ejecución real (Spectral, Redocly, Prism, pytest sobre PostgreSQL, Ruff) y contra los escenarios del dossier. Los defectos encontrados y las decisiones del equipo constan en docs/04-registro-ia.md.

Datos: no se ingresaron datos personales ni credenciales.

Referencias

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2

Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley.

Cucumber. (s. f.). *Gherkin reference*. https://cucumber.io/docs/gherkin/reference/

International Organization for Standardization & International Electrotechnical Commission. (2008). *Information technology — Guideline for the evaluation and selection of CASE tools* (ISO/IEC 14102:2008). ISO.

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2017). *Systems and software engineering — Software life cycle processes* (ISO/IEC/IEEE 12207:2017). ISO.

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2018). *Systems and software engineering — Life cycle processes — Requirements engineering* (ISO/IEC/IEEE 29148:2018). ISO.

North, D. (2006). Introducing BDD. *Better Software*. https://dannorth.net/introducing-bdd/

OpenAPI Generator. (s. f.). *OpenAPI Generator documentation*. https://openapi-generator.tech/docs/

OpenAPI Initiative. (s. f.). *OpenAPI Specification (v3.x)*. Linux Foundation. https://spec.openapis.org/oas/latest.html

OWASP Foundation. (s. f.). *OWASP Top 10*. https://owasp.org/www-project-top-ten/

Stoplight. (s. f.). *Spectral and Prism documentation*. https://docs.stoplight.io